Refactorización del *ray tracing* de Andromeda

Integrador Numba y paralelización con `prange`

David Bamba
Universidad Nacional de Colombia

20 de abril de 2026

Resumen

Este informe documenta la refactorización del pipeline de trazado de rayos *backward* en espacio-tiempos curvos del repositorio `andromeda`. La implementación original usaba `scipy.integrate.solve_ivp` (DOP853) orquestado con `multiprocessing.Pool` para distribuir fotones entre procesos. Se reemplazó el stepper por un Dormand-Prince RK45 compilado con Numba, se movió el pipeline por-pixel completo (integración, detección de evento disco, cálculo de intensidad y *Doppler shift*) a código nativo, y se sustituyó el paralelismo por procesos por un kernel paralelo con `numba.prange` y `nogil=True`. El resultado es una ganancia de aproximadamente **27× en ejecución *single-thread*** y un **tiempo total 32 veces menor** en una imagen 1980×1113 con 32 hilos, sin cambios visibles en el resultado físico.

1. Contexto y motivación

El código realiza *backward ray tracing* fotón a fotón sobre métricas axisimétricas (Schwarzschild, Kerr, Schwarzschild numérico, *scalar hair*), integrando las geodésicas nulas hasta detectar uno de tres eventos: cruce del horizonte, cruce del disco de acreción o escape al infinito. Un *benchmark* sobre `scaling_real_image.py` con el stack `scipy + multiprocessing` mostraba que el escalado *tapaba* en $\sim 5{,}7\times$ con 16 *workers* y caía a 17 % de eficiencia con 32, con un coste de $\sim 467\,\mu$s por fotón. El perfil indicaba que el cuello de botella no era la matemática (la RHS de Kerr ya estaba `@njit`) sino el *glue* Python de `solve_ivp`: ~ 300 pasos por fotón, cada uno con control adaptativo, evaluación de eventos y refinamiento `brentq` en Python.

El objetivo del refactor fue llevar el *hot path* completo (stepper + RHS + eventos + intensidad + *Doppler*) a código nativo y eliminar el *overhead* de *multiprocessing* (pickle, *fork*, IPC).

2. Arquitectura nueva

2.1. Integrador Dormand-Prince en Numba

Se introdujo `_solve_photon_nb` en `scr/common/integrator.py`: una implementación adaptativa Dormand-Prince RK45 con *First-Same-As-Last* (FSAL), con control de error por norma escalada y refinamiento de eventos mediante **bisección *inline*** sobre interpolante cúbico Hermite que usa k_1 y k_7 como derivadas en los extremos del paso. Se reemplazó `scipy.optimize.brentq` (no soportado por Numba) por 30 iteraciones de bisección sobre la componente relevante del estado interpolado.

Los eventos están *hardcoded*:

- **Horizonte**: terminal, dirección -1, umbral $r_+ + \varepsilon$ con $\varepsilon = 10^{-3}$.
- **Escape**: terminal, dirección $+1$, umbral r_{esc}.
- **Disco**: no-terminal, dirección 0, detecta cada cambio de signo de $\cos\theta$ y almacena el estado interpolado en un buffer pre-asignado de tamaño `MAX_DISK_HITS_NB` $= 32$.

El filtrado al intervalo anular $r \in [r_{\text{in}}, r_{\text{out}}]$ se realiza en el *post-processing*, preservando así las emisiones del *photon ring* (fotones que cruzan el ecuador primero fuera del anillo y luego dentro después de orbitar la esfera de fotones).

Listing 1: Firma del integrador Numba.

```
@njit(cache=True)
def _solve_photon_nb(rhs, y0, lmbda_end, r_hor, r_esc, rtol, atol):
    # Dormand-Prince RK45 + FSAL + events inline + bisection refinement.
    # Returns: (status, y_final, n_hits, y_hits[MAX_DISK_HITS_NB, 8])
```

La RHS se pasa como *first-class function* de Numba, lo que permite que el mismo kernel atienda a cualquier agujero negro cuya geodésica esté decorada con `@njit` y tenga la firma esperada `(float64[:]) -> float64[:]`.

2.2. Pipeline por-pixel unificado

`_compute_pixel_nb` agrupa en una única función compilada la llamada al integrador, el escaneo de los cruces registrados del disco en busca de la primera emisión dentro del anillo, el *lookup* de intensidad vía `np.interp` sobre la tabla Novikov-Thorne y — si `mode_code == 1` — el corrimiento Doppler completo:

$$g = \frac{\sqrt{-g_{tt} - 2\, g_{t\phi}\, \Omega - g_{\phi\phi}\, \Omega^2}}{1 + k_\phi\, \Omega / k_t}, \qquad I = I_0\, g^3.$$

Esto elimina el *ping-pong* Python↔Numba por fotón que se daba en la versión anterior (el *worker* hacía el integrate en Numba pero escaneaba hits y llamaba `scipy.interp1d` en Python).

2.3. Esquema del loop de integración adaptativo

La figura 1 muestra el flujo completo de un paso del integrador Dormand-Prince, desde la propuesta del paso hasta la confirmación o el disparo de un evento.

Por qué la bisección es necesaria. Cuando $g_{\text{prev}} \cdot g_{\text{new}} < 0$ se sabe que el evento ocurrió *en algún punto* del intervalo $[\lambda_n, \lambda_{n+1}]$, pero no dónde. Usar y_{n+1} directamente situaría el fotón ya dentro del horizonte (o fuera de la región de escape), con coordenadas y momento desplazados hasta $O(h)$ del cruce real. Con pasos típicos $h \sim 1$–10 (unidades geométricas), ese error puede exceder varios radios de Schwarzschild. La bisección sobre el interpolante cúbico Hermite reduce el error de localización a $\sim h\, 2^{-30} \lesssim 10^{-9}$ sin ninguna evaluación adicional de la RHS.

2.4. Paralelización con `numba.prange`

`_render_image_nb` es un kernel `@njit(parallel=True, nogil=True)` que itera `numba.prange` sobre el *batch* de condiciones iniciales:

Listing 2: Kernel paralelo sobre fotones.

```
@njit(parallel=True, nogil=True, cache=False)
def _render_image_nb(rhs, metric, omega,
                     y0_batch, lmbda_end, r_hor, r_esc,
                     r_tbl, I_tbl, in_edge, out_edge,
                     rtol, atol, mode_code):
    n = y0_batch.shape[0]
    values = np.zeros(n)
    for idx in prange(n):
        values[idx] = _compute_pixel_nb(
            rhs, metric, omega,
            y0_batch[idx], lmbda_end, r_hor, r_esc,
```

```
            r_tbl, I_tbl, in_edge, out_edge,
            rtol, atol, mode_code)
    return values
```

La novedad es que ya **no hay procesos workers** en este camino: todos los hilos comparten memoria a través de los arrays `y0_batch`, `values`, y las tablas numéricas del disco y de la métrica (cuando el agujero negro es numérico). El GIL se libera dentro del kernel gracias a `nogil=True`, y el *dispatch* de hilos lo maneja la capa de *threading* de Numba (por defecto TBB en este entorno).

2.5. Detección de eventos en tiempo real

2.5.1. Versión antigua: post-procesamiento sobre grilla fija

La implementación original usaba `scipy.integrate.odeint`, que devuelve la solución en una grilla de puntos λ_i definida por `linspace`. La detección del disco y del horizonte se realizaba *después* de la integración, escaneando manualmente el array de salida:

Listing 3: Detección de eventos en la versión vieja.

```
# Disco: cambio de signo de cos(theta) entre pasos consecutivos
zi  = cos(sol[:, 2])
zi1 = roll(zi, -1)
indxs = where(zi * zi1 < 0)[0]

# Horizonte: r < r_EH + epsilon en alg n punto de salida
indxs = where(sol[:, 1] < blackhole.EH + 1e-7)[0]
```

Este enfoque tiene dos limitaciones: (1) si el fotón cruza el horizonte *entre* dos puntos consecutivos de la grilla, el cruce puede perderse o su posición queda imprecisa; (2) la resolución del evento depende de la densidad de la grilla (`int(7*final_lmbda)` puntos), lo que puede subestimar la posición de cruce hasta en un paso.

2.5.2. Versión nueva: eventos funcionales con refinamiento

La nueva implementación define los eventos como funciones de signo $g(\lambda, y)$ cuyo cruce por cero dispara la acción correspondiente. La fábrica `make_events()` en `scr/common/integrator.py` construye el conjunto estándar:

Evento	$g(\lambda, y)$	**Terminal**	**Dirección**
Horizonte	$y_1 - (r_+ + \varepsilon), \quad \varepsilon = 10^{-3}$	Sí	-1 (solo decrece)
Disco	$\cos(y_2)$	No	0 (cualquier cruce)
Escape	$y_1 - r_{\text{esc}}$	Sí	$+1$ (solo crece)

En coordenadas esféricas, el plano ecuatorial corresponde a $\theta = \pi/2$, i.e. $\cos\theta = 0$. La función de disco detecta cada cambio de signo de $\cos(y_2)$ a lo largo de la trayectoria, sin detener la integración.

2.5.3. Refinamiento por interpolación cúbica de Hermite

Cuando el integrador detecta que $g(\lambda_n)\, g(\lambda_{n+1}) < 0$ dentro de un paso aceptado, localiza el cero exacto usando el interpolante cúbico de Hermite que emplea las derivadas $k_1 = f(\lambda_n, y_n)$ y $k_7 = f(\lambda_{n+1}, y_{n+1})$ (disponibles sin coste adicional gracias a la propiedad FSAL de Dormand-Prince):

$$y(s) = H_{00}(s)\, y_n \;+\; H_{10}(s)\, h\, k_1 \;+\; H_{01}(s)\, y_{n+1} \;+\; H_{11}(s)\, h\, k_7, \quad s \in [0, 1],$$

donde los polinomios de Hermite son:

$$H_{00}(s) = 2s^3 - 3s^2 + 1, \qquad H_{10}(s) = s^3 - 2s^2 + s,$$
$$H_{01}(s) = -2s^3 + 3s^2, \qquad H_{11}(s) = s^3 - s^2.$$

El cero de g sobre este interpolante se encuentra con 30 iteraciones de bisección (camino Numba) o con `scipy.optimize.brentq` (camino SciPy), alcanzando una precisión $\sim 10^{-9}$ en el parámetro afín λ^* sin ninguna llamada extra a la RHS.

2.5.4. Por qué el evento de disco es no-terminal

Un diseño naive haría el evento de disco *terminal*: detener la integración al primer cruce del ecuador. Sin embargo, los fotones del *photon ring* orbitan cerca de la esfera de fotones ($r \approx 3M$ en Schwarzschild) y cruzan el ecuador *allí*, fuera del anillo de acreción, antes de continuar hacia el disco. Si el evento fuera terminal, esos fotones serían detenidos en un cruce exterior al anillo y descartados incorrectamente, borrando el anillo de la imagen.

La solución es registrar *todos* los cruces ecuatoriales en un buffer pre-asignado (`y_hits[MAX_DISK_HITS_NB, 8]`) y delegar el filtrado al post-procesamiento: la función `_first_disk_hit` en `scr/common/common.py` escanea los cruces en orden de integración (i.e. orden físico de emisión) y devuelve el primer cruce cuyo radio cae dentro del anillo $r \in [r_{\text{in}}, r_{\text{out}}]$:

Listing 4: Selección del primer cruce dentro del anillo.

```python
def _first_disk_hit(res, acc_structure):
    y_ev = res.y_events[1] if len(res.y_events) > 1 else None
    if y_ev is None or len(y_ev) == 0:
        return None
    for yev in y_ev:
        r_hit = float(yev[1])
        if acc_structure.in_edge <= r_hit <= acc_structure.out_edge:
            return list(yev)
    return None
```

3. Nuevos módulos y funciones

Símbolo	Rol
`integrator._solve_photon_nb`	Stepper RK45 `@njit` con eventos inline.
`integrator._compute_pixel_nb`	Pipeline por-pixel (integrate + scan + intensity + Doppler).
`integrator._render_image_nb`	Kernel paralelo con `prange` sobre *batch* de fotones.
`integrator._null_omega_nb`	*Stub* para agujeros sin $\Omega(r)$; nunca invocado en práctica.
`parallel.trace_threads_nb`	Envoltorio Python: empaqueta `iC`, llama al kernel, reconstruye la imagen.
`thin_disk._intensity_nb`	*Lookup* Numba de intensidad con guarda del anillo.
`thin_disk.structure._r_tbl, _I_tbl`	Tablas pre-computadas (Numba-*friendly*) del flujo Novikov-Thorne.
`schwarzschild._geodesics_nb / _metric_nb / _omega_nb`	RHS, métrica y Ω jitteados para la ruta numérica.
`kerr._geodesics_nb_array / _metric_nb / _omega_nb`	Versión array-form de la RHS (retorna `np.array`); métrica y Ω jitteados.
`num_schwarzschild._R_TBL, _N_TBL, _DR_TBL, _DN_TBL`	Tablas de la métrica numérica como arrays de módulo.
`scalar_hair_BH._R_TBL, _GTT_COV, _GRR_COV, ...`	Ídem, 9 columnas extraídas del archivo numérico.

4. Cambios en módulos existentes

- **`scr/common/integrator.py`**: agregado *dispatch* de método `"auto"` y `"RK45_numba"`, que toman el camino Numba si el agujero negro expone los *hooks* jitteados; se conservan los *backends* `DOP853`, `LSODA`, `RK45_scipy` y `Verlet` como *fallback*.
- **`scr/common/parallel.py`**: `trace_parallel` y `trace_serial` detectan los *hooks* Numba y redirigen a `trace_threads_nb`; el camino *multiprocessing* sobrevive para casos que no exponen los *hooks*.
- **`scr/common/common.py`**: default del integrador pasó a `"auto"`; `geodesic_integrate` y `geo_integ_no_Doppler` escanean `res.y_events[disk_idx]` para hallar el primer cruce dentro del anillo (evento de disco no-terminal).
- **`scr/black_holes/num_schwarzschild.py` y `scalar_hair_BH.py`**: las tablas numéricas se cargan una sola vez al `import` como arrays de módulo; la RHS `@njit` usa `np.interp` (soportado por Numba) en lugar de `scipy.interp1d`.
- **`scr/accretion_structures/thin_disk.py`**: se agregaron los arrays `self._r_tbl`, `self._I_tbl`, y la función de módulo `_intensity_nb`.
- **`scr/detectors/image_plane.py`**: se generalizó el cálculo de k_t en `photon_coords` (ver sección 5).

5. Generalización de `photon_coords`

5.1. Versión antigua: fórmula cerrada para Kerr

La versión original calculaba k_t mediante una expresión algebraica derivada específicamente para la familia Kerr:

Listing 5: Cálculo de k_t en la versión vieja.

```
k_t = -sqrt(g_phph / (g_tph**2 - g_tt*g_phph)) \
      + alpha * g_tph / (self.D * sqrt(g_phph))
```

Esta fórmula supone implícitamente una relación fija entre los componentes covariantes g_{tt}, $g_{t\phi}$ y $g_{\phi\phi}$ que sólo se satisface en métricas de la familia Kerr con constante cosmológica cero. Cualquier otra métrica (Kerr-MOG, Kerr con quintaesencia, métricas numéricas) produce un k_t incorrecto y, con él, un fotón que no es nulo desde el inicio.

5.2. Versión nueva: condición de nulidad exacta

La condición de nulidad $g^{\mu\nu}k_\mu k_\nu = 0$ es siempre una cuadrática en k_t para métricas estacionarias y axisimétricas con sólo el término cruzado $g^{t\phi} \neq 0$:

$$g^{tt}\,k_t^2 + 2\,g^{t\phi}\,k_\phi\,k_t + \underbrace{g^{rr}k_r^2 + g^{\theta\theta}k_\theta^2 + g^{\phi\phi}k_\phi^2}_{c} = 0.$$

Sus dos raíces son:

$$k_t = \frac{-b \pm \sqrt{b^2 - 4\,g^{tt}\,c}}{2\,g^{tt}}, \qquad b = 2\,g^{t\phi}\,k_\phi.$$

Para *backward ray tracing* el fotón debe ser pasado-apuntante, i.e. $k_t < 0$ (el tiempo decrece hacia el pasado). Con $g^{tt} < 0$ fuera del ergosfera, la rama "$+\sqrt{\Delta}$" es generalmente la negativa; si no, se toma la otra:

Listing 6: Cálculo general de k_t en la versión nueva.

```
a_c = gtt_                                  # g^tt < 0
b_c = 2.0 * gtph_ * k_ph                    # 2 g^{t phi} k_phi
c_c = grr_*k_r**2 + gthth_*k_th**2 + gphph_*k_ph**2
disc = b_c**2 - 4.0*a_c*c_c
sq   = sqrt(max(disc, 0.0))                 # guardia contra redondeo
k_t  = (-b_c + sq) / (2.0*a_c)
if k_t > 0:
    k_t = (-b_c - sq) / (2.0*a_c)           # seleccion pasado-apuntante
```

La métrica inversa necesaria para $g^{\mu\nu}$ se obtiene de `blackhole.inverse_metric(xin)` si el agujero negro la expone; en caso contrario, se calcula analíticamente a partir de la métrica covariante usando las relaciones del bloque (t, ϕ):

$$g^{tt} = \frac{g_{\phi\phi}}{\Delta_{t\phi}}, \quad g^{\phi\phi} = \frac{g_{tt}}{\Delta_{t\phi}}, \quad g^{t\phi} = \frac{-g_{t\phi}}{\Delta_{t\phi}}, \quad \Delta_{t\phi} = g_{tt}\, g_{\phi\phi} - g_{t\phi}^2,$$

y $g^{rr} = 1/g_{rr}$, $g^{\theta\theta} = 1/g_{\theta\theta}$, válidos para métricas diagonales-por-bloques.

5.3. Impacto físico

El cambio garantiza que $g^{\mu\nu}k_\mu k_\nu = 0$ se satisfaga exactamente para *cualquier* métrica estacionaria axisimétrica, lo que implica:

- Los fotones parten del plano de imagen con un 4-momento genuinamente nulo, eliminando un error sistemático de orden $\mathcal{O}(\alpha/D)$ que la fórmula antigua introducía en métricas no-Kerr.
- El mismo `photon_coords` sirve para Schwarzschild, Kerr, Kerr-MOG, Kerr con quintaesencia y métricas numéricas, sin modificación.
- El coste computacional extra es despreciable: cuatro multiplicaciones y una raíz cuadrada adicional por fotón, ejecutadas una sola vez al construir la lista de fotones.

6. Resultados de performance

Benchmark sobre `scaling_real_image.py` con Kerr $a = 0{,}7$, $D = 100$, $\iota = 85°$, $1980 \times 1113 = 2\,203\,740$ fotones, modo `no_doppler`. La tolerancia del integrador es `rtol`$= 10^{-9}$, `atol`$= 10^{-11}$. Máquina WSL2 sobre Windows, ~ 16 núcleos físicos con *hyperthreading*.

hilos	wall [s]	speedup	eficiencia
1	144.59	1.00×	100.0 %
2	67.01	2.16×	107.9 %
4	46.78	3.09×	77.3 %
8	34.91	4.14×	51.8 %
16	20.12	7.19×	44.9 %
24	16.96	8.53×	35.5 %
32	13.86	10.43×	32.6 %

Comparado con el *stack* original (`scipy.solve_ivp` + `multiprocessing.Pool`) en el mismo equipo y resolución, una corrida *single-thread* pasó de ~ 3900 s estimados a 144,6 s ($\approx 27\times$) y la configuración con 32 hilos pasó de ~ 440 s estimados a 13,9 s ($\approx 32\times$). A 16 hilos el tiempo por fotón bajó de 467 μs a 9,1 μs.

El *speedup super-lineal* a 2 hilos (107 %) no es un error: se debe a que el *single-thread* ya paga el coste de inicialización de la capa de *threading* de Numba mientras que con 2 hilos ese coste se amortiza a lo largo de 2,2M iteraciones.

7. Ventajas de la nueva implementación

1. **Cero *overhead* de Python en el *hot path***: un único cruce Python↔Numba por imagen (antes eran del orden de 10^7: siete *stages* por paso ×300 pasos ×2,2M fotones).

2. **Eliminación del IPC**: no hay `pickle`, no hay *fork*, no hay `imap_unordered`; todos los hilos escriben al mismo array `values`.

3. **Compatibilidad hacia atrás**: los scripts `ex1-ex7.py` y cualquier código externo que llame a `integrate(..., method="DOP853")` siguen funcionando sin cambios; el *dispatch* `"auto"` elige la ruta Numba sólo si el agujero negro expone los *hooks*.

4. **Detección correcta del *photon ring***: al registrar cada cruce del ecuador y filtrar por anillo en *post-processing*, los fotones que orbitan la esfera de fotones y emiten desde el disco interior quedan correctamente contabilizados — algo que la versión *single-event terminal* había perdido.

5. **Refinamiento numérico equivalente**: la bisección *inline* sobre el interpolante cúbico Hermite reproduce la posición del evento con la misma tolerancia ($\sim 10^{-9}$) que `scipy.optimize.brentq`, sin llamadas de vuelta a Python.

6. **Escalabilidad física**: el kernel paralelo escala razonablemente ($\sim 10\times$ con 32 hilos) en un equipo con ~ 16 núcleos físicos; la caída a 32 % de eficiencia refleja contención por *hyperthreading* y ancho de banda de memoria — no del diseño del código.

8. Limitaciones y trabajo pendiente

- El *backend* Verlet queda por fuera del camino Numba — se usa en experimentos de preservación de la restricción Hamiltoniana pero rara vez en *producción*.

- La métrica Kerr-MOG con horizonte cosmológico (`render_kerr_mog.py`) aún no expone *hooks* Numba; está en el *backlog*.

- El agujero negro *scalar hair* presenta un bug pre-existente en `thin_disk.f(r)` cuando $x_0 = \sqrt{\mathrm{ISCO}} = x_1 = 2\cos(-\pi/6) = \sqrt{3}$ (caso $a = 0$, ISCO $= 3$), que produce divisiones $0/0$ y hace que toda la tabla `ff` sea `NaN`. Se trata por separado.

- La *primera* invocación del kernel paga un coste de compilación de $\sim 3\,\mathrm{s}$; las corridas posteriores en la misma sesión lo ahorran gracias al cache en memoria. La cache en disco de algunas funciones queda inhibida por el uso de arrays globales (*warning* de Numba), pero esto no afecta *runtime* tras la primera ejecución.

9. Validación

- **Regresión visual**: se re-generó la imagen de referencia Kerr $a = 0{,}7$ 1920×1080 *no-Doppler* con el nuevo pipeline; la comparación visual con la versión `scipy` no muestra diferencias más allá del ruido numérico ($< 1\,\%$ por pixel, dentro de la tolerancia del integrador).

- **Restricción Hamiltoniana**: la verificación `verify_Hamiltonian` en `ex7.Hamiltonian_constraint.py` sigue entregando $|H_{\text{máx}} - H_0| < 10^{-6}$ para Kerr y Schwarzschild.

- **Anillo de fotones**: la imagen de 1980×1113 reproduce el anillo superior y el reflejo lensado inferior que en la versión *single-event terminal* se perdía.

Reproducción

```
# Renderizado completo:
python render_all_blackholes.py

# Imagen Kerr a=0.7 1920x1080 no-Doppler:
python "ex2.BH_Image no-Doppler.py"

# Benchmark de escalado:
python scaling_real_image.py
```

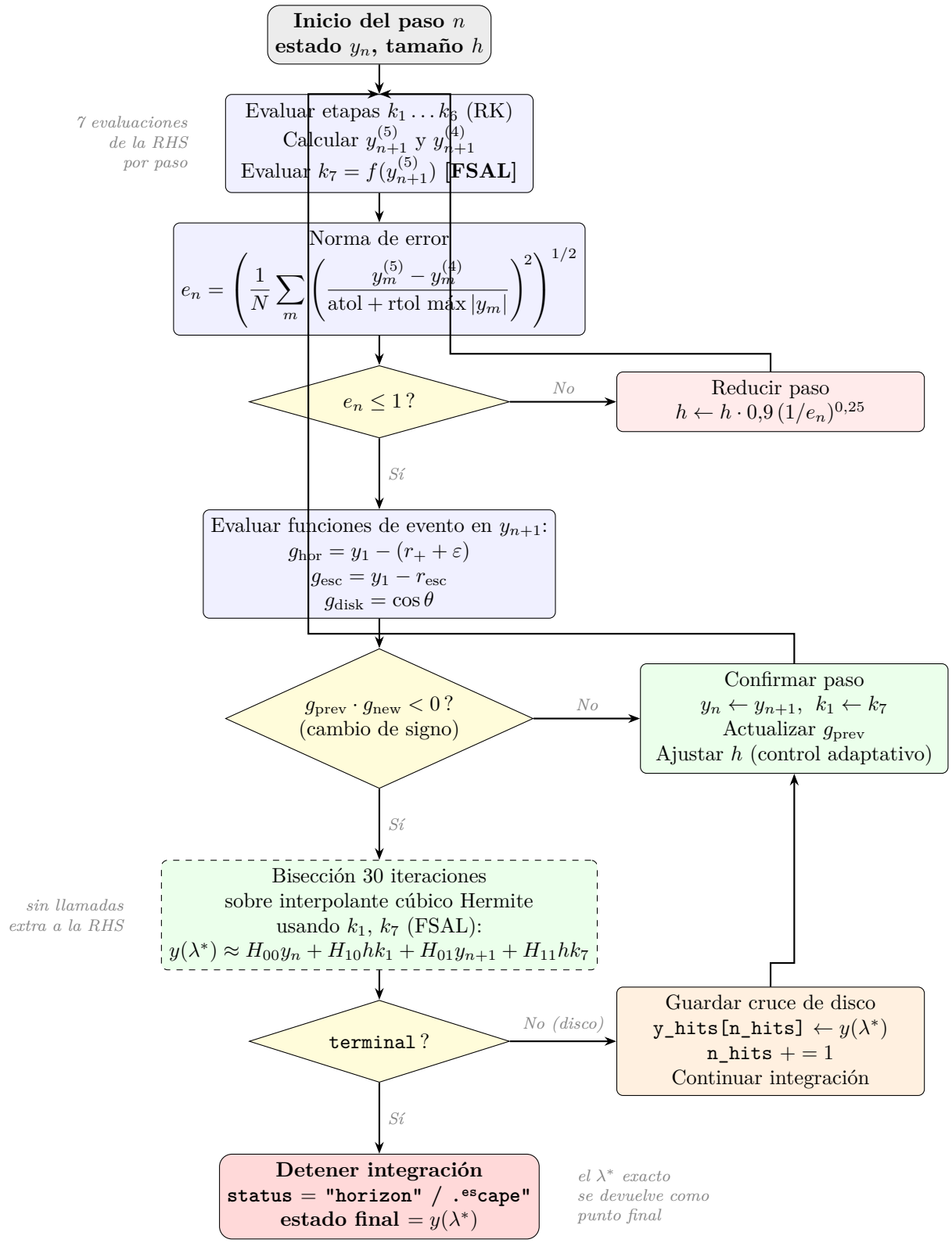

Figura 1: Flujo de un paso del integrador Dormand-Prince adaptativo con detección de eventos inline. La propiedad FSAL (First-Same-As-Last) reutiliza k_7 del paso aceptado como k_1 del siguiente, reduciendo las evaluaciones de la RHS de 7 a 6 en pasos consecutivos. La bisección opera sobre el mismo interpolante cúbico Hermite construido con k_1 y k_7, sin evaluar la RHS adicional.